

DiskAnalyzer

Extension Fix Guide

Root-cause analysis and actionable fixes for the
blank-screen / crash issue diagnosed from Log_2026-06-14.txt

EXTENSION

DiskAnalyzerExtension 1.2.0

PLATFORM

PowerToys CmdPal 0.100 .NET 10, x64

REPORT DATE

June 14, 2026

PREPARED FOR

Thet

What the Log Reveals

The log file **Log_2026-06-14.txt** captured 17 Command Palette sessions. The good news: your extension loads and registers successfully every time — Loaded 1 command(s) and 0 band(s) appears consistently. The crash or blank-screen happens the moment a command is *invoked*, not at load time, which is why it is silent and gives no visible error.

Startup Time Comparison (from log)

Your extension consistently takes **3–5× longer** than comparable extensions to activate:

VisualStudioForCmdPal

~450 ms ✓

SparseApp (built-in)

~390 ms ✓

DiskAnalyzerExtension

1,600 – 2,025 ms ✗ TOO SLOW



Root Cause: Synchronous Disk I/O on the COM Host Thread

The CmdPal host has a strict activation timeout. When your extension blocks the COM thread for ~2 seconds doing disk work, the host kills the process and shows nothing. One session in the log (10:41:48) confirms the host completely dropped the extension then auto-restarted it — that is the crash-and-re-appear cycle you are seeing.



One Fast Entry Proves the Fix Will Work

At 11:23:00 a single restart completed in just 11 ms because cached data was already available. This confirms your code path is correct — you just need to shift the heavy work off the COM thread.

Fix 2 — Async-First GetItems() Pattern

```

public sealed class DiskAnalyzerPage : ListPage
{
    private DriveInfo[]? _cachedDrives;

    // Constructor - fast, no I/O
    public DiskAnalyzerPage()
    {
        Name = "Disk Analyzer";
        Icon = new IconInfo("\uE9F9");
    }

    // Returns immediately with placeholder, loads async
    public override IListItem[] GetItems()
    {
        if (_cachedDrives is null)
        {
            _ = LoadDrivesAsync(); // fire-and-forget
            return [ new ListItem(new NoOpCommand())
            {
                Title = "Scanning drives...",
                Subtitle = "Results will appear shortly"
            }
            ];
        }
        return BuildDriveItems(_cachedDrives);
    }

    // All heavy work runs on a thread-pool thread
    private async Task LoadDrivesAsync()
    {
        var drives = await Task.Run(() =>
            DriveInfo.GetDrives().Where(d => d.IsReady).ToArray());
        _cachedDrives = drives;
        RaiseItemsChanged(0); // CmdPal re-calls GetItems()
    }
}

```

Fix 3 — Disk-Specific Pitfalls to Guard Against

Trap	Symptom	Severity	Fix
DriveInfo.GetDrives() with disconnected network drive	Hangs 30–90 s, host kills extension	CRITICAL	Filter with <code>.where(d=>d.IsReady)</code>
Accessing <code>drive.VolumeLabel</code> on unready drive	Throws <code>IOException</code> , extension crashes	CRITICAL	Wrap every property access in try/catch
<code>Directory.GetFiles(path, "**", AllDirectories)</code>	Blocks for minutes on large drives	HIGH	Use <code>EnumerateFiles()</code> inside <code>Task.Run()</code> with <code>CancellationToken</code>
Recursive folder size on <code>C:\</code> without depth limit	Never finishes; host kills after timeout	HIGH	Add <code>maxDepth</code> limit (e.g. 3 levels) or use WMI query
No try/catch around <code>GetItems()</code>	Any exception silently kills the extension	MEDIUM	Wrap entire method body; return error <code>ListItem</code>

Defensive Drive Iteration Pattern

```
private static IEnumerable<DriveInfo> GetSafeDrives()
{
    foreach (var drive in DriveInfo.GetDrives())
    {
        bool ready = false;
        try { ready = drive.IsReady; }
        catch { continue; } // unreadable drive – skip

        if (ready) yield return drive;
    }
}
```

Fix 4 — Always Wrap `GetItems()` in Try/Catch

If any unhandled exception escapes `GetItems()`, the COM host silently terminates your extension. The palette shows nothing and gives no error message. This wrapping is **mandatory** for disk-access extensions:

```

public override IListItem[] GetItems()
{
    try
    {
        if (_cachedDrives is null)
        {
            _ = LoadDrivesAsync();
            return [LoadingPlaceholder()];
        }
        return BuildDriveItems(_cachedDrives);
    }
    catch (Exception ex)
    {
        // Show visible error card – never crash silently
        return [ new ListItem(new NoOpCommand())
        {
            Title    = "Error loading drives",
            Subtitle = ex.Message
        }
        ];
    }
}

```

Fix 5 — Reload Extensions After Every Deploy

After deploying a new build from Visual Studio, Command Palette caches the old extension metadata. You **must** manually reload — otherwise your changes won't appear even after a successful deploy.



Reload Procedure (do this after every deploy)

Open Command Palette → type 'reload' → select 'Reload Command Palette extensions'. Wait ~2 seconds, then search for your extension name again.



Still Not Showing After Reload?

Go to PowerToys Settings → Command Palette → Extensions, find DiskAnalyzer, toggle it Off then On, then reload again.

Complete Fix Checklist

Work through these in order — each one can independently cause the blank/crash behaviour:

- ☐ Remove all DriveInfo / directory scanning from the page constructor
- ☐ Remove all disk I/O from CommandsProvider constructor and TopLevelCommands()
- ☐ Return a loading placeholder from GetItems() immediately when cache is null
- ☐ Run all disk scanning inside Task.Run() on a background thread
- ☐ Call RaiselItemsChanged(0) after background scan completes
- ☐ Filter drives with .Where(d => d.IsReady) before accessing any property
- ☐ Wrap every per-drive property access in try/catch
- ☐ Wrap the entire GetItems() body in try/catch, return error ListItem
- ☐ Add a depth limit or CancellationToken to any recursive directory scan

- ☐ Run 'Reload Command Palette extensions' inside CmdPal after every deploy
- ☐ Confirm startup time drops to <500 ms in the CmdPal log after changes

SECTION 8

Verifying the Fix in the Logs

After applying the fixes and redeploying, check the new log at:

```
%LOCALAPPDATA%\Microsoft\PowerToys\CmdPal\Logs
```

Look for your extension entry. A successful fix will show:

```
[Info] TopLevelCommandManager.cs::TryStartExtensionAsync
      Started extension DiskAnalyzerExtension_1.2.0.0 in 312 ms
[Info] CommandProviderWrapper.cs::InitializeCommands
      Loaded 1 command(s) and 0 band(s) in 28 ms
```

V Success Criteria

Startup time under 500 ms and no Error entries for DiskAnalyzer means the fix is working. The async scan fires after the page opens, and RaiseltemsChanged populates the results.